Computing an MSA with Genetic Algorithm

REPORT

Pedro Galego

Professor: André Lamúrias

2025-10-23

Abstract

In this project, I implemented a Genetic Algorithm to optimize Multiple Sequence Alignments of protein sequences using the sum-of-pairs scoring scheme with the **BLOSUM62** substitution matrix and gap penalties.

Each individual in the population represents a complete MSA, where each row corresponds to an align protein sequence with gaps.

This report serves to describe the design, implementation and outlining how each component (Initialize the population, Score, Selection, Crossover, Mutation and Termination) contributes to the overall optimization process.

Faculdade de Ciências e Tecnologia da Universidade Nova de Lisboa

Mestrado em Engenharia Informática

BioInformática



Table of Contents

1 Initialize the population **1**

2 Score **1**

3 Selection **2**

4 Crossover **3**

5 Mutation **3**

6 Termination **4**

7 Credits **4**

1 Initialize the population

The initialization phase defines the starting population for the Genetic Algorithm, where each individual represents a possible MSA. On my implementation, each MSA is generated by introducing random gaps in front of the sequence and at the end of the sequence. This ensures that every sequence within an MSA has the same total length while maintaining diversity across individuals in the population. I do this as follows:

1. Retrieve the sequences to build each MSA as a list and the population size to know how much MSA's do I have to create.
2. Verify the longest sequence from all the sequences in the **cytochromes.fa** file.
3. Add a random number to the longest sequence between **MIN-ADDED-SIZE** and **MAX-ADDED-SIZE**, so this way we know how much gaps we have to add to each sequence.
4. Create each MSA by going through each sequence and adding random number of gaps to the right between the max length of a sequence minus the sequence size, and to the left, the rest that wasn't added to the right side.
5. Then I create a list containing all the sequences from a single MSA, and add that list to the list of lists where all the MSA's are being stored.

2 Score

The evaluate the score of each MSA, I used the **sum-of-pairs** scoring method. This metric measures how well residues are aligned across all sequences by summing pairwise substitution scores for every column in the alignment.

1. First I examine each column of the MSA sequentially
2. Then, for each column, the algorithm compares all possible pairs of sequences in that column:
    - If either residue is a gap, the gap penalty is applied
    - If both position contain amino acids, the algorithm retrieves the corresponding score from the **BLOSUM62** substituion matrix

3. The total score of the MSA is obtained by summing all pairwise comparisons across every column.

The final result represents the **fitness** of the alignment.

3 Selection

In the Selection phase, is where we determine which MSA's are carried over or "reproduced" to form the next generation. This step ensures that individuals with high scores (fitness) have a greater chance of influencing the future generations.

1. The top 50 percent of the MSA's with the highest score are directly appended to the next generation list.

2. I then normalize the score between every MSA such that, the lowest score has a score of 0. This was done to ensure that all the MSA's can have a probability attached to them for the crossover algorithm.

3. The remaining 50 percent of the next generations is created through a probabilistic reproduction, where the probability of selecting an MSA as a parent is proportional to the it's score. I calculate a cumulative probability to facilitate random choosing of MSA based on their probability of being chosen.

4. Now each new offspring is generated either by:

   - Crossover -> where we combine the parents based on the cumulative probability of each of them.
   - Mutation -> where we remove all gaps of a random sequence inside the MSA and append them at the beginning of the sequence.

   This was done by a 50/50 chance on each of the remaining 50 percent of the sequences.

The resulting generation maintains the same size as the previous one.

4 Crossover

The crossover operator is responsible for combining the characteristics of two parents MSA's to create a new offspring, promoting the exchange of structural features between alignments. In this project, the crossover was implemented exactly as the procedure described in file **Project 1 preparation.pdf**.

5 Mutation

The mutation operator introduces small random changes in a sequence of each new MSA forming the new generation. This is done to preserve population. In my implementation, the process is to remove all gaps from a sequence inside an MSA and append them to the beginning of the sequence to preserve it's length.

1. First select randomly one sequence from the MSA
2. Then remove all gaps from within it
3. Add the new gaps at the beginning of the sequence to preserve it's length
4. Reinsert the new mutated sequence into the MSA

Now I will give a small working example to show how the mutation operation works:

```
# This is an example of the mutation operator
align_exemple = [
    "ATC-G-G-TT",
    "A-CCGC-ATC",
    "A-CCG--ATC"
]
print(mutation(align_exemple))
```

When we run this code, the result we get is

```
['ATC-G-G-TT', 'A-CCG--ATC', '--ACCGCATC']
```

In this example

1. The algorithm randomly choose a sequence to mutate, which in this case would have been the second sequence.
2. It then popped the sequence out of the MSA.
3. Removed the gaps from within the sequence.
4. Added them back at the front of the sequence.
5. And finally, it reinserted the sequence into the MSA.

The mutation operator contributes by maintaining diversity in the MSA's which ensures that the algorithm continues to generate new types of alignments that were not previously present in the population.

6 Termination

After a predefined number of generations, the Genetic Algorithm terminates. When this happens, the algorithm selects the best individual based on the highest fitness score and prints it as the final result. This MSA represents the algorithm's best approximation of an optimal MSA according to the defined scoring strategy.

7 Credits

This is the LaTeX template (**Version 1.1**) from the DH Lab of the University of Basel.